

AI Career Accelerator

AI Powered Career Development Platform

Complete Professional Project Documentation

Production-Ready SaaS Application Specification

MERN Stack | Google Gemini AI | Vercel | Render | MongoDB Atlas

Prepared as official product and engineering documentation

Format: Editable Microsoft Word (.docx)

Date: 01 July 2026

Team Members

Arman Mahetar | 20230905050004

Backend Development, API Development, Database Integration

Hardik Makwana | 20230905050009

Backend Development, Testing, Quality Assurance

Krunal Rathod | 20230905050032

Frontend Development, AI Integration, Deployment

Certificate Page

This is to certify that the project documentation titled 'AI Career Accelerator - AI Powered Career Development Platform' presents the architecture, product design, implementation blueprint, and deployment-ready technical specification of a SaaS application developed using React.js, Node.js, Express.js, MongoDB, and Google Gemini AI. The document has been prepared in a professional software engineering format and reflects a production-oriented approach covering frontend, backend, database, AI integration, analytics, administration, security, deployment, and quality practices.

Prepared By	Authorized Review
Project Team	Signature / Seal

Declaration

We hereby declare that the documentation titled 'AI Career Accelerator' is an original and professionally prepared technical project document describing the architecture, workflows, design system, modules, APIs, and operational considerations of an AI-powered career development platform. The content has been compiled for software product documentation purposes and represents a consolidated description of solution strategy, implementation approach, engineering best practices, and production deployment readiness.

Prepared By	Authorized Review
Project Team	Signature / Seal

Acknowledgement

This documentation was shaped through the combined perspectives of software architecture, technical writing, UI/UX design, and product management. The project team acknowledges the importance of disciplined engineering practices, thoughtful user experience design, secure system construction, and structured documentation in transforming a concept into a production-grade SaaS platform. Special appreciation is extended to everyone who contributes to technical review, testing rigor, deployment discipline, and continuous product improvement.

Prepared By	Authorized Review
Project Team	Signature / Seal

Abstract

AI Career Accelerator is a unified career development platform that addresses a fragmented student placement-preparation journey through AI-assisted resume creation, applicant-tracking analysis, job-description matching, mock interviews, performance feedback, skill-gap assessment, and personalized coaching. The platform is designed as a modern SaaS solution with responsive frontend delivery, scalable backend services, AI-enhanced workflows, cloud-managed infrastructure, and role-based administration. This document provides complete professional documentation for the platform, including product rationale, system architecture, route structure, user workflows, UI specification, database design, API documentation, deployment guidance, testing strategy, security controls, and future scalability directions.

Prepared By	Authorized Review
Project Team	Signature / Seal

Table of Contents

Note: The table of contents is inserted as an automatic Word field. If your Word client does not refresh it immediately, right-click the table and choose Update Field.

Update field to display contents.

Introduction

AI Career Accelerator is conceived as a digital career enablement system for students and early professionals who require structured, measurable, and AI-assisted support while preparing for internships, campus recruitment, and full-time placements. Instead of treating resume creation, interview practice, job matching, and career planning as isolated activities, the platform unifies them under one authenticated, analytics-driven workspace.

From a product strategy perspective, the platform solves both convenience and effectiveness. Users are not only given tools to create and upload career artifacts, but also given interpretation, guidance, and progression logic. This means the application is not a static document builder; it acts as an interactive decision-support system that continuously converts user data into action plans, feedback, and next-step recommendations.

From an engineering perspective, the system is designed as a production-ready SaaS application using React on the frontend, Express and Node.js on the backend, MongoDB as the persistence layer, and Google Gemini as the AI intelligence provider. The architecture supports modular feature growth, operational security, analytics tracking, and role-based control needed for long-term maintainability.

Problem Statement

Students preparing for placements typically depend on disconnected tools for resume creation, resume review, applicant tracking analysis, interview preparation, skill planning, and career guidance. This fragmented ecosystem leads to duplicated effort, inconsistent data, poor progress visibility, and a lack of continuity between one preparation activity and the next.

Manual review cycles are often slow, subjective, and difficult to scale. A student may create a resume in one tool, test it in another, compare it to a job description in a third service, and seek mock interview support elsewhere. None of these systems share context, making it difficult to generate personalized recommendations based on the complete profile and performance history of the user.

The central problem, therefore, is the absence of an integrated, intelligent platform that can combine content creation, analysis, simulation, coaching, and performance analytics into a coherent career-development workflow optimized for student outcomes and administrative oversight.

Existing System

In the current market, students generally rely on standalone resume builders, generic job portals, basic ATS keyword tools, publicly available interview question banks, and informal mentoring channels such as social communities or ad-hoc chat support. These tools may individually solve a narrow problem but rarely provide lifecycle continuity.

Most existing systems focus on one feature category. Resume builders emphasize templates but not job-fit diagnostics. ATS analyzers focus on keyword density but do not integrate user profile history. Interview tools may simulate questions but fail to convert performance signals into a structured roadmap. Career guidance portals often remain content repositories rather than interactive systems.

As a result, existing solutions do not provide a consolidated intelligence layer that can understand the user across documents, targets, interview performance, and long-term skill development goals. The lack of integration reduces both product value and operational efficiency.

Limitations of Existing System

Existing systems are limited by fragmented experiences, context loss across tools, low personalization depth, weak analytics continuity, and poor alignment with specific placement outcomes. Data generated in one activity is seldom reused to improve later recommendations or workflows.

Another major limitation is the inconsistent quality of feedback. Static validators can identify formatting issues but cannot explain trade-offs, propose tailored improvements, or connect resume shortcomings to likely interview risk. Similarly, mock interview systems often provide generic scoring without actionable development guidance.

Operationally, many tools lack a secure administration layer, role-based data governance, usage analytics, and production-grade deployment practices, which makes them unsuitable as institutional platforms or scalable SaaS applications.

Proposed System

The proposed system is an end-to-end AI-powered career development platform that allows a user to move from onboarding to resume creation, analysis, job alignment, interview practice, coaching, and roadmap execution without leaving the application context. Every module contributes to a cumulative understanding of the user's career readiness.

The platform integrates AI services at decisive moments where interpretation and personalization deliver measurable value: generating resume content, parsing uploaded resumes into editable structures, analyzing ATS compatibility, matching resumes against target job descriptions, producing interview questions, evaluating answers, and constructing personalized learning roadmaps.

Because the application is designed as a SaaS product rather than a prototype, the proposed system also includes secure authentication, stateful dashboards, admin operations, cloud asset handling, analytics instrumentation, deployment workflows, and maintainable folder structures for future scale.

Objectives

The primary objective of AI Career Accelerator is to improve placement readiness through an integrated AI-enabled platform that converts isolated preparation tasks into a guided user journey. The system seeks to reduce friction, improve decision quality, and create measurable progress visibility for both users and administrators.

A secondary objective is to create a technically sound and commercially viable SaaS foundation. This includes modular frontend architecture, secure backend APIs, extensible data design, cost-aware AI orchestration, cloud-ready deployment, and operational practices aligned with production software delivery.

The final objective is to establish a platform capable of personalization at scale, where user-specific data can inform better resume suggestions, job-fit scores, interview readiness, roadmap recommendations, and career-coaching interactions over time.

Scope

The scope of this project covers the design, implementation blueprint, and documentation of a web-based SaaS application for students and early-career professionals. Functional scope includes authentication, user profile management, resume generation, resume parsing, ATS analysis, job-description comparison, mock interview simulation, feedback generation, roadmap planning, coaching, analytics, and administration.

The technical scope includes responsive frontend design, RESTful backend APIs, MongoDB-based persistence, JWT-based authentication, Gemini-based AI services, asset storage integration, cloud deployment, security controls, testing strategy, and operational monitoring considerations.

Out of scope for the current release are native mobile applications, offline-first mode, enterprise single sign-on, multi-language localization, and institution-specific LMS integrations, though the architecture intentionally leaves room for future extension in these areas.

Technology Stack

The technology stack has been selected to balance developer productivity, rapid iteration, cloud readiness, and the ability to support an interactive AI-enhanced user experience. The chosen stack also aligns with common SaaS engineering patterns, simplifying hiring, onboarding, and long-term maintenance.

React.js and Tailwind CSS provide a fast and scalable UI delivery model, while Express.js and Node.js provide a lightweight but capable backend layer for route handling, middleware composition, and external API integration. MongoDB provides a flexible schema model that fits evolving AI-driven documents and interaction records.

Google Gemini AI serves as the intelligence engine across content generation and analysis workflows, while Vercel, Render, MongoDB Atlas, and ImageKit provide production-friendly hosting, persistence, and media-management capabilities.

Frontend Stack

Layer	Technology	Professional Rationale
Framework	React.js	Component-driven single-page application architecture with reusable UI and route-based navigation.
Styling	Tailwind CSS	Utility-first styling system for rapid, consistent, responsive interface implementation.
Component Library	ShadCN UI	Accessible, modern UI primitives for dialogs, forms, cards, dropdowns, tables, and sheets.
Routing	React Router	Client-side route management with protected route enforcement and nested layouts.
Data Fetching	React Query	Caching, background

		synchronization, mutation management, and request-state control.
Forms	React Hook Form	High-performance form state management with validation and error binding.
HTTP Client	Axios	Centralized API communication layer with interceptors for JWT handling and retry control.

Backend Stack

Layer	Technology	Professional Rationale
Runtime	Node.js	Event-driven JavaScript runtime for scalable I/O-heavy SaaS workloads.
Framework	Express.js	REST API layer, middleware orchestration, authentication control, and error handling.
Database	MongoDB	Document-oriented persistence layer for user profiles, resumes, analytics, and conversational records.
Authentication	JWT Authentication	Secure access and refresh token strategy for session continuity and API protection.
Password Security	bcrypt	Password hashing and verification with configurable salt rounds.

Cloud and AI Stack

Layer	Technology	Professional Rationale
Media Storage	ImageKit	Managed asset upload, transformation, optimization, and delivery for profile and

		resume resources.
Database Hosting	MongoDB Atlas	Managed cloud database platform with backups, monitoring, and scaling.
AI Layer	Google Gemini API	Prompt-driven intelligence for resume writing, analysis, interviewing, roadmap generation, and coaching.
Frontend Deployment	Vercel	Global deployment platform for React frontend with preview environments and CDN acceleration.
Backend Deployment	Render	Managed deployment for Express services with environment management and observability.

Tip: This stack is intentionally optimized for rapid iteration, cost-aware deployment, and modular feature expansion.

System Architecture

The platform follows a layered web architecture composed of a presentation layer, API orchestration layer, domain service layer, persistence layer, and external service integration layer. Each layer is intentionally separated to simplify scaling, reduce coupling, and preserve maintainability.

On the frontend, React renders route-based experiences and manages local interaction state, while React Query coordinates server communication and cache consistency. The backend exposes REST endpoints, validates requests, authenticates users, coordinates business rules, invokes Gemini prompts, and persists results to MongoDB.

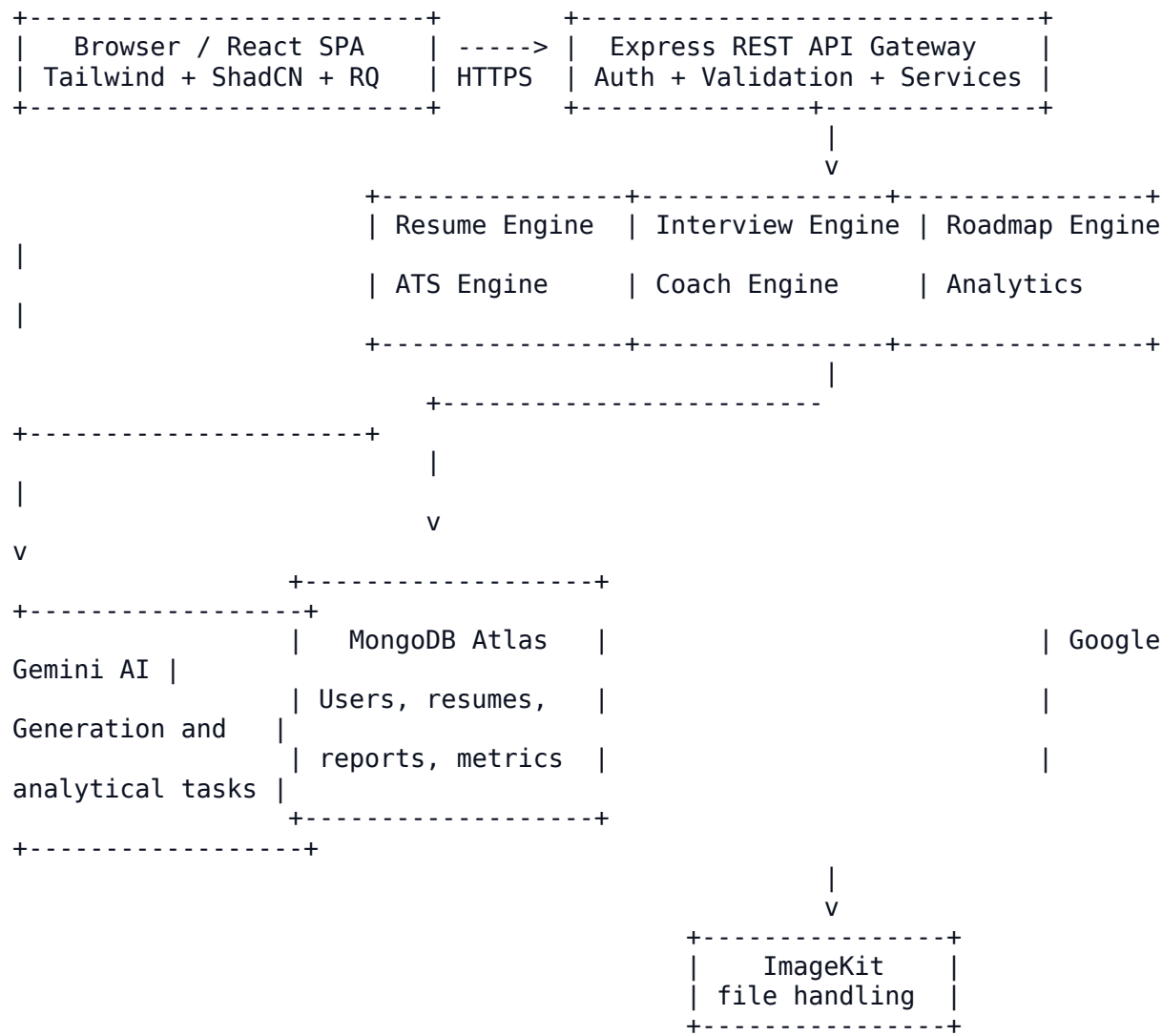
External services include Google Gemini for generative and analytical tasks, ImageKit for file and media handling, MongoDB Atlas for managed persistence, and deployment platforms for runtime availability. This separation allows the core application logic to remain independent from hosting concerns.

Architecture Layers

Layer	Core Components	Responsibilities
Presentation Layer	React, Tailwind, ShadCN UI	Renders responsive user interfaces, protected views, forms, charts, and interaction flows.
Application Layer	Express controllers and services	Validates requests, enforces access rules, coordinates business workflows, and composes API responses.
AI Orchestration Layer	Gemini prompt services	Generates structured content, analysis, coaching responses, and recommendation artifacts.
Persistence Layer	MongoDB Atlas	Stores user profiles, resumes, reports, roadmaps, analytics events, and administrative logs.

Infrastructure Layer	Vercel, Render, ImageKit	Handles deployment, edge delivery, runtime hosting, and asset storage/optimization.
----------------------	--------------------------	---

Architecture Diagram



Note: The architecture separates interactive rendering from domain orchestration so that AI-dependent workflows can evolve without destabilizing the user interface layer.

Project Modules

The system is organized into domain-focused modules so that each capability can evolve independently while still contributing to a shared user profile and analytics layer. This modular design supports maintainability, testing isolation, and phased feature rollout.

Modules are grouped around the user lifecycle: onboarding, content creation, analysis, simulation, guidance, reporting, and administration. This ensures that feature boundaries reflect product behavior rather than arbitrary technical grouping.

Module	Primary Objective	Operational Summary
[ICON] Resume Builder	Create structured resumes	Section-based editor, live preview, AI bullet generation, template selection, export and draft versioning.
[ICON] Resume Parser	Convert uploaded resume into editable data	File ingestion, text extraction, section mapping, confidence checks, editable reconstruction.
[ICON] AI Resume Writer	Improve content quality	Role-aware summary generation, achievement reframing, keyword insertion, tone refinement.
[ICON] ATS Analyzer	Evaluate ATS readiness	Keyword analysis, structure checks, formatting review, score generation, actionable recommendations.
[ICON] Resume vs JD Match	Measure alignment with target role	Similarity scoring, gap detection, requirement mapping, prioritized edit suggestions.
[ICON] Mock Interview	Simulate interviews	Question generation, answer capture, timer support, transcript storage, session progression.
[ICON] Interview Feedback	Turn answers into coaching output	Competency scoring, strength and weakness

		analysis, communication advice, improvement plan.
[ICON] Skill Gap Analysis	Identify capability deficiencies	Role requirement analysis, missing-skill detection, proficiency mapping, training priorities.
[ICON] Career Roadmap	Create milestone-driven growth plan	Timeline generation, milestone grouping, action steps, progress tracking, resource recommendations.
[ICON] AI Career Coach	Provide contextual guidance	Conversational assistance, saved insights, role-based advice, application strategy support.
[ICON] Dashboard Analytics	Summarize progress	Readiness snapshots, trend charts, activity history, module usage reporting.
[ICON] Admin Panel	Control and monitor platform	User administration, analytics oversight, moderation controls, audit visibility.

Complete Feature List

The platform delivers a comprehensive feature portfolio designed for production use rather than demonstration-only scenarios. Features cover user self-service, AI-assisted content generation, decision support, analytics, administrative oversight, and cloud-friendly operational support.

Each feature has been designed to either reduce friction in the user journey, improve the quality of career decisions, or provide measurable visibility into progression and platform usage. Together, they form a connected product system instead of a collection of disconnected utilities.

Domain	Feature	Professional Explanation
Authentication	Email signup and login	Secure account access with validation, hashing, JWT issuance, and verification workflow.
Profile	Career profile management	Centralized user identity, education, experience, goals, and preference data.
Resume	Resume creation and editing	Guided forms, AI content help, version management, and export readiness.
Resume	Resume parsing	Upload existing documents and convert them into editable structured data.
Analysis	ATS scoring	Resume diagnostics with keyword, formatting, and section-level evaluation.
Analysis	JD match	Role-fit comparison between resume and job description with actionable gap findings.
Interview	Mock session generation	Role-specific question generation with answer capture and timing logic.
Interview	Feedback report	Detailed evaluation across clarity, relevance, confidence, and

		technical accuracy.
Roadmap	Skill-gap based planning	Transforms missing capabilities into milestones, timelines, and learning actions.
Coach	Career coaching chat	AI assistant for ongoing guidance on roles, applications, interviews, and improvement.
Analytics	Dashboard reporting	Tracks scores, milestones, activity, and engagement history.
Admin	Platform governance	User management, content moderation, system metrics, and audit review.

User Workflow

The platform workflow is intentionally progressive. Users begin by creating an account and building a career profile, then move into resume creation or upload, followed by analysis, job targeting, interview preparation, and long-term planning. This sequence mirrors how real placement preparation unfolds.

A key design principle is continuity. Output from one stage feeds the next stage. Resume quality affects ATS scoring, ATS output affects job-match readiness, interview performance affects the roadmap, and user interactions influence coaching recommendations and dashboard analytics.

Progressive User Workflow

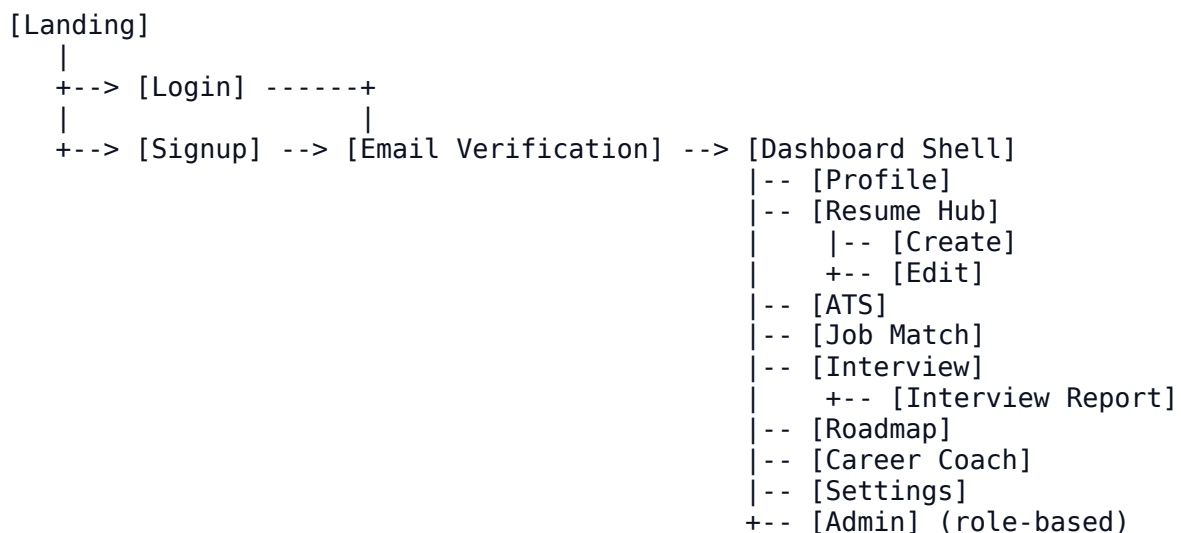
1. Create account and verify identity through the authentication flow.
2. Complete profile information to improve personalization quality.
3. Create or parse a resume and store it as an editable asset.
4. Run ATS analysis to identify structural and keyword issues.
5. Compare resume with target job descriptions to understand role fit.
6. Practice AI mock interviews and review detailed performance feedback.
7. Generate a personalized career roadmap based on observed gaps.
8. Use the AI career coach for ongoing refinement, strategy, and decision support.
9. Track progress through dashboard analytics and iterate continuously.

Website Navigation Flow

Navigation is organized around a clear public-to-private product journey. Public pages establish value and acquire users, while authenticated pages form a structured workspace for execution, feedback, and progress tracking.

Protected routes are grouped under the dashboard shell so users experience consistent navigation, breadcrumb context, and action shortcuts across all career modules. The result is a predictable information architecture that lowers cognitive load and supports fast task switching.

Navigation Sequence Diagram



Tip: All private routes should share a persistent application shell containing sidebar navigation, breadcrumbs, theme controls, and notification access.

Complete Route Structure

The route model is organized to separate public acquisition screens from authenticated productivity workflows. Private screens are grouped under a dashboard-centered experience so that the user keeps orientation, context, and persistent access to the platform's most important actions.

Route	Access	Explanation
/	Public	Landing page presenting product value proposition, feature summary, trust signals, and conversion CTAs.
/login	Public	Secure sign-in screen with email and password authentication, error handling, and recovery links.
/signup	Public	Registration flow for new users with validation, password policy messaging, and role capture.
/dashboard	Protected	Primary workspace showing analytics cards, quick actions, recent activity, and module shortcuts.
/profile	Protected	User profile management page for personal data, education, experience summary, and preferences.
/resume	Protected	Resume management hub listing drafts, imported files, status tags, and resume-related actions.
/resume/create	Protected	Guided resume builder with sections, AI assistance, form controls, and live preview.
/resume/edit	Protected	Editable resume

		workspace used to refine imported or previously saved resumes.
/ats	Protected	ATS analysis page for scanning resumes, scoring compatibility, and surfacing optimization insights.
/job-match	Protected	Resume versus job description comparison page with similarity scoring and gap explanation.
/interview	Protected	Mock interview module for question generation, answer capture, timing, and progression tracking.
/interview/report	Protected	Detailed interview report page displaying transcript, evaluation, strengths, and improvement plan.
/roadmap	Protected	Personalized roadmap page showing skill gaps, recommended milestones, and learning actions.
/career-coach	Protected	Conversational AI coach interface for contextual career guidance and follow-up support.
/settings	Protected	Application preferences, security settings, notification controls, and account management tools.
/admin	Admin	Administrative console for user management, platform analytics, moderation, and configuration oversight.

/

Access Type: Public. Landing page presenting product value proposition, feature summary, trust signals, and conversion CTAs. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/login

Access Type: Public. Secure sign-in screen with email and password authentication, error handling, and recovery links. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/signup

Access Type: Public. Registration flow for new users with validation, password policy messaging, and role capture. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/dashboard

Access Type: Protected. Primary workspace showing analytics cards, quick actions, recent activity, and module shortcuts. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/profile

Access Type: Protected. User profile management page for personal data, education, experience summary, and preferences. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/resume

Access Type: Protected. Resume management hub listing drafts, imported files, status tags, and resume-related actions. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/resume/create

Access Type: Protected. Guided resume builder with sections, AI assistance, form controls, and live preview. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/resume/edit

Access Type: Protected. Editable resume workspace used to refine imported or previously saved resumes. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/ats

Access Type: Protected. ATS analysis page for scanning resumes, scoring compatibility, and surfacing optimization insights. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/job-match

Access Type: Protected. Resume versus job description comparison page with similarity scoring and gap explanation. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/interview

Access Type: Protected. Mock interview module for question generation, answer capture, timing, and progression tracking. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/interview/report

Access Type: Protected. Detailed interview report page displaying transcript, evaluation, strengths, and improvement plan. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/roadmap

Access Type: Protected. Personalized roadmap page showing skill gaps, recommended milestones, and learning actions. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/career-coach

Access Type: Protected. Conversational AI coach interface for contextual career guidance and follow-up support. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/settings

Access Type: Protected. Application preferences, security settings, notification controls, and account management tools. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

/admin

Access Type: Admin. Administrative console for user management, platform analytics, moderation, and configuration oversight. This route should be implemented with explicit loading, permission, and error states so that navigation remains predictable under both normal and degraded network conditions.

Page-by-Page UI Design

The UI specification is structured around clarity, actionability, and confidence. Every page must communicate purpose immediately, minimize unnecessary interaction cost, and expose meaningful system feedback in loading, error, and empty conditions. The design system should preserve consistency across cards, forms, buttons, tables, charts, and modal interactions.

Landing Page

The Landing Page is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Acquire visitors, establish credibility, communicate differentiators, and convert traffic into account creation.
Core Components	Hero section; Primary CTA; Feature grid; How-it-works strip; Testimonials; FAQ accordion; Footer navigation
Cards	Feature cards; Success metric cards; Trust badges; Pricing or plan teaser cards
Buttons	Get Started; Login; View Features; Watch Demo
Forms	Newsletter or lead capture form
Tables	Optional comparison table for platform benefits versus fragmented tools
Charts	Simplified placement readiness progress visualization or impact metric snapshot
Primary Actions	Navigate to signup; Scroll to features; Open demo; Read FAQs
Loading State	Skeleton blocks for hero metrics and delayed testimonial content.
Error State	Fallback message for failed marketing data or testimonial retrieval.
Empty State	Graceful static content when dynamic promotional blocks are unavailable.
Responsive Design	Hero converts from two-column to

	stacked layout with touch-friendly CTAs.
Dark Mode	Dark hero surface, muted section dividers, and accessible contrast on call-to-action buttons.

Note: Icon placeholders such as [ICON: Landing Page] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Authentication

The Authentication is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Register and authenticate users securely while keeping friction low and validation clear.
Core Components	Auth card; Brand panel; Inline validation messages; Password strength indicator; Social proof area
Cards	Login card; Signup card; Benefits side panel
Buttons	Create Account; Sign In; Forgot Password; Resend Verification
Forms	Signup form; Login form; Forgot password form; OTP or email verification input
Tables	None in the default flow
Charts	None
Primary Actions	Submit credentials; Toggle password visibility; Switch auth mode; Resend verification
Loading State	Button-level loading states, disabled inputs, and progress spinner during request submission.
Error State	Credential mismatch, duplicate email, expired verification link, and rate-limit feedback.
Empty State	Not applicable; helper text replaces absent optional data.

Responsive Design	Single-column card layout on mobile with large tap targets and short labels.
Dark Mode	Elevated auth card on dark background with high-contrast input borders.

Note: Icon placeholders such as [ICON: Authentication] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Dashboard

The Dashboard is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Provide a consolidated overview of readiness, pending actions, and cross-module progress.
Core Components	Top summary cards; Activity timeline; Action center; Progress widgets; Recommendation panel
Cards	Resume score card; ATS score card; Interview score card; Roadmap progress card
Buttons	Create Resume; Run ATS; Start Interview; Open Roadmap
Forms	Quick filter form for time range or target role
Tables	Recent activity table; Saved resume list
Charts	Progress trend chart; Module usage chart; Skill distribution chart
Primary Actions	Jump to modules; Review recommendations; Open reports; Track milestones
Loading State	Dashboard skeleton cards, chart placeholders, and animated table rows.
Error State	Partial data fallback with retry controls and informational banner.
Empty State	Onboarding state with guided next

	steps when no user activity exists.
Responsive Design	Cards collapse into stacked layout and charts shift to simplified views on small screens.
Dark Mode	Dark analytics cards with subtle borders, muted gridlines, and bright accent callouts.

Note: Icon placeholders such as [ICON: Dashboard] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Resume Builder

The Resume Builder is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Enable users to create structured, ATS-friendly resumes with AI-assisted content suggestions.
Core Components	Section sidebar; Form workspace; Live preview pane; AI suggestion drawer; Template selector
Cards	Resume section cards; Suggestion cards; Completion status card
Buttons	Generate with AI; Save Draft; Preview; Export; Import Resume
Forms	Personal details; Education; Experience; Projects; Skills; Certifications; Achievements
Tables	Version history table; Imported data review table
Charts	Completion progress bar or readiness meter
Primary Actions	Add section; Reorder entries; Apply suggestion; Save version
Loading State	Field-level spinners and preview skeleton during AI generation.
Error State	Validation summaries, failed AI response messaging, and draft save recovery options.

Empty State	Starter template cards and smart examples for first-time users.
Responsive Design	Preview pane moves below form area; sticky action bar remains accessible.
Dark Mode	Monochrome editing canvas with blue accent actions and legible form contrast.

Note: Icon placeholders such as [ICON: Resume Builder] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

ATS

The ATS is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Assess resume compatibility with applicant tracking expectations and provide optimization guidance.
Core Components	Upload area; Score meter; Keyword analysis panel; Section diagnostics; Recommendation list
Cards	ATS score card; Keyword density card; Formatting issue card; Improvement summary card
Buttons	Analyze Resume; Re-run Analysis; Apply Suggestions; Download Report
Forms	Resume upload form; Target role input; Optional job description field
Tables	Keyword match table; Section issue table
Charts	Score gauge; Keyword category chart
Primary Actions	Upload file; Inspect issue; Send suggestion to resume builder
Loading State	Analysis progress indicator with staged status messaging.
Error State	Unsupported format message, failed parse warning, and AI timeout recovery.
Empty State	Illustrated empty state explaining how ATS analysis improves application success.

Responsive Design	Score card prioritizes top-level findings first on mobile.
Dark Mode	Dark analysis dashboard with colored severity indicators remaining accessible.

Note: Icon placeholders such as [ICON: ATS] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Job Match

The Job Match is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Compare a resume against a specific job description and highlight alignment and gaps.
Core Components	Input split view; Match score panel; Gap summary; Recommended edits; Evidence mapping
Cards	Overall match card; Missing skill card; Experience alignment card; Priority action card
Buttons	Compare Now; Improve Resume; Save Match; Export Summary
Forms	Resume selector; Job description textarea; Role metadata fields
Tables	Matched keyword table; Gap table; Experience relevance table
Charts	Match radar or category comparison chart
Primary Actions	Paste JD; Run comparison; Open edit mode; Track target roles
Loading State	Split-panel skeleton and deferred recommendation rendering.
Error State	Parsing failure banner, empty JD warning, and unsupported resume structure feedback.
Empty State	Guided instructions prompting the user to select a resume and paste a

	job description.
Responsive Design	Comparison panels stack vertically while keeping score summary sticky.
Dark Mode	Dark comparison surfaces with clear highlighted diffs and readable table borders.

Note: Icon placeholders such as [ICON: Job Match] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Interview

The Interview is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Simulate structured mock interviews and evaluate candidate responses through AI analysis.
Core Components	Interview setup panel; Question card; Answer input area; Timer; Progress tracker
Cards	Question card; Performance summary card; Confidence indicator card
Buttons	Start Interview; Next Question; Submit Answer; End Session
Forms	Role selection; Difficulty selector; Interview type selector; Answer input or transcript capture
Tables	Past interview sessions table
Charts	Per-round score chart; Confidence trend chart
Primary Actions	Generate questions; Record answer; Review transcript; Open report
Loading State	Question-generation loader and answer-evaluation progress stepper.
Error State	Microphone denial, transcript issue, AI response delay, and save failure messaging.
Empty State	Pre-session setup state showing best-practice tips and recommended

	interview types.
Responsive Design	Large recording controls and stacked transcript view for tablet and mobile use.
Dark Mode	Focused dark practice environment with bright timer and minimal distractions.

Note: Icon placeholders such as [ICON: Interview] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Roadmap

The Roadmap is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Translate identified skill gaps into a structured progression plan with milestones and timelines.
Core Components	Milestone timeline; Skill-gap panel; Learning recommendations; Progress tracker; Deadline cards
Cards	Goal summary card; Milestone card; Priority skill card; Completion card
Buttons	Generate Roadmap; Mark Complete; Adjust Goal; Export Plan
Forms	Target role input; Time availability selector; Current skill self-assessment
Tables	Milestone plan table; Recommended resources table
Charts	Progress chart; Skill category chart
Primary Actions	Generate milestones; Track completion; Revise timeline; Open linked learning path
Loading State	Milestone skeletons and staged roadmap generation status updates.
Error State	Insufficient profile data warning and roadmap regeneration fallback.
Empty State	Prompt-led onboarding state explaining how roadmap quality

	improves with profile completeness.
Responsive Design	Timeline switches to card sequence with collapsible milestone details.
Dark Mode	Dark roadmap canvas with accent milestone connectors and high-contrast status pills.

Note: Icon placeholders such as [ICON: Roadmap] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Career Coach

The Career Coach is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Provide contextual AI coaching for career planning, application strategy, communication, and next-step decision making.
Core Components	Chat panel; Context drawer; Suggested prompts; Conversation history; Pinned insights
Cards	Suggested prompt card; Actionable advice card; Saved insight card
Buttons	Ask Coach; Save Insight; Generate Plan; Clear Context
Forms	Chat input; Context selector; Goal or role preference inputs
Tables	Saved coaching topics table
Charts	Optional advice-category usage chart
Primary Actions	Ask question; Attach profile context; Save recommendation; Continue previous thread
Loading State	Typing indicator, token-generation progress hint, and message placeholder bubbles.
Error State	Rate-limit notice, AI service timeout, and context-size reduction advice.
Empty State	Starter prompt suggestions for resumes, interviews, role targeting,

	and networking.
Responsive Design	Chat area expands full width and context drawer becomes a bottom sheet on mobile.
Dark Mode	Messaging interface optimized for long reading sessions with restrained accent highlights.

Note: Icon placeholders such as [ICON: Career Coach] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Profile

The Profile is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Store and maintain user identity, education, experience, preferences, and readiness metadata.
Core Components	Profile header; Editable information sections; Avatar upload; Career preferences panel
Cards	Personal details card; Education card; Experience card; Target role card
Buttons	Edit Profile; Save Changes; Upload Avatar; Sync Resume Data
Forms	Personal info form; Education form; Experience form; Preferences form
Tables	Achievement list table; Application history summary table
Charts	Profile completeness meter
Primary Actions	Update details; Change preferences; Upload image; View activity
Loading State	Section placeholders and optimistic save indicators.
Error State	Image upload error, validation warnings, and unsaved-change alerts.
Empty State	Profile completion prompts with contextual examples.
Responsive Design	Sections convert to stacked cards with persistent save controls.
Dark Mode	Dark profile cards with muted dividers

	and clearly labeled inputs.
--	-----------------------------

Note: Icon placeholders such as [ICON: Profile] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Settings

The Settings is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Control account security, notification preferences, theme options, and connected service settings.
Core Components	Tabbed settings layout; Security panel; Notification controls; Theme toggles; Danger zone
Cards	Password card; Session management card; Theme settings card; Notification settings card
Buttons	Update Password; Enable 2FA; Save Preferences; Delete Account
Forms	Password update form; Notification preferences form; Privacy settings form
Tables	Active sessions table; Audit activity summary table
Charts	None by default
Primary Actions	Rotate password; Logout other sessions; Toggle email alerts; Switch theme
Loading State	Control-level loaders and save-status banners.
Error State	Incorrect current password feedback and notification-service failure warnings.
Empty State	Default preference state with explanatory helper text.
Responsive Design	Tabbed layout becomes accordion-based on smaller devices.
Dark Mode	Low-glare settings canvas with clear

	separation between safe and destructive actions.
--	--

Note: Icon placeholders such as [ICON: Settings] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Admin

The Admin is designed as a production-ready interface surface rather than a purely visual mockup. It should align with the product goal of making career-preparation tasks understandable, guided, and outcome-driven.

Design Dimension	Professional Specification
Purpose	Provide operators with oversight of users, analytics, moderation events, and platform health.
Core Components	Admin summary cards; User management table; Content moderation tools; Audit panel; Health widgets
Cards	Active user card; AI usage card; Error rate card; Pending issue card
Buttons	View User; Suspend Account; Export Report; Refresh Metrics
Forms	Search and filter controls; User role update form; Announcement form
Tables	User list table; Audit log table; Support issue table
Charts	Usage trend chart; Conversion chart; Module adoption chart
Primary Actions	Review accounts; Update roles; Inspect events; Track adoption
Loading State	Data-grid skeletons and chart placeholders for heavy datasets.
Error State	Permission errors, partial metrics load warnings, and admin action failure alerts.
Empty State	Guided empty state for new deployments without operational data.
Responsive Design	Priority metrics stay visible while tables compress into stacked records.
Dark Mode	Dark console with strong emphasis on alerts, statuses, and audit readability.

Note: Icon placeholders such as [ICON: Admin] can be used during wireframing and replaced later with a consistent Lucide or custom icon set.

Complete User Journey

The end-to-end user journey is designed to lead a first-time visitor from discovery to measurable career improvement. Each stage is expected to generate both user value and product context so the next stage becomes more relevant and personalized.

Landing
|
Create Account
|
Verify Email
|
Login
|
Dashboard
|
Create Resume
|
ATS Analysis
|
Job Match
|
Interview
|
Feedback
|
Roadmap
|
Career Coach
|
Profile
|
Logout

10. The visitor lands on the marketing page, understands the value proposition, and chooses to create an account.
11. The registration flow captures identity, target role, and security credentials while enforcing validation rules.
12. Email verification confirms account ownership and unlocks the protected workspace.
13. After login, the dashboard highlights next actions, usually profile completion or resume creation.
14. The user creates a resume or parses an existing one, then analyzes it for ATS readiness.
15. The user compares the resume with target job descriptions to improve relevance before applying.

16. Mock interview sessions simulate likely hiring scenarios and produce actionable feedback.
17. The platform translates observed weaknesses into a roadmap and supplements it with contextual coaching.
18. Profile and settings remain available for continuous refinement until the user logs out or returns later.

Frontend Folder Structure

The frontend codebase follows a feature-oriented structure supported by shared components and platform-level utilities. This balance prevents monolithic page files while keeping reusable design elements easy to discover and maintain.

Recommended Folder Tree

```
src/
|-- api/
|   |-- axiosClient.js
|   |-- authApi.js
|   |-- resumeApi.js
|   |-- interviewApi.js
|-- app/
|   |-- router.jsx
|   |-- providers.jsx
|-- assets/
|-- components/
|   |-- common/
|   |-- dashboard/
|   |-- forms/
|   |-- charts/
|   |-- layout/
|-- features/
|   |-- auth/
|   |-- profile/
|   |-- resume/
|   |-- ats/
|   |-- jobMatch/
|   |-- interview/
|   |-- roadmap/
|   |-- careerCoach/
|   |-- admin/
|-- hooks/
|-- lib/
|-- pages/
|-- store/
|-- styles/
|-- utils/
|-- App.jsx
|-- main.jsx
```

Folder Explanation

Folder	Professional Explanation
api	Encapsulates Axios clients and domain-specific request functions so UI layers remain independent from raw endpoint definitions.
app	Contains root routing, provider composition, query client setup, and

	application bootstrapping concerns.
assets	Stores static visual assets such as logos, illustrations, and non-generated theme resources.
components	Houses reusable presentational and composite UI components shared across multiple pages or features.
features	Implements domain-based frontend modules such as resume, interview, roadmap, and coaching workflows.
hooks	Provides custom React hooks for business logic reuse, API interaction abstractions, and UI utilities.
lib	Contains framework adapters, helper configuration, schema utilities, and third-party integration wrappers.
pages	Defines route-level page components that compose layout, domain features, and route guards.
store	Optional global state utilities for app-wide settings, lightweight UI state, or auth hydration.
styles	Theme tokens, Tailwind composition helpers, and style-level customizations beyond utility classes.
utils	Pure helper functions such as formatters, validators, transformers, and shared constants.

Backend Folder Structure

The backend is organized to separate transport concerns, business logic, persistence, validation, and AI prompt assets. This structure improves testability and allows multiple developers to work in parallel without excessive merge conflicts or logic duplication.

Recommended Folder Tree

```
server/
|-- src/
|   |-- config/
|   |-- controllers/
|   |-- middlewares/
|   |-- models/
|   |-- routes/
|   |-- services/
|   |-- repositories/
|   |-- validations/
|   |-- prompts/
|   |-- utils/
|   |-- jobs/
|   |-- constants/
|   |-- app.js
|   |-- server.js
|-- tests/
|-- docs/
|-- package.json
```

Folder Explanation

Folder	Professional Explanation
config	Centralizes environment loading, database initialization, cloud service wiring, and runtime options.
controllers	Translates HTTP requests into service calls and standard API responses without embedding heavy business logic.
middlewares	Implements authentication, authorization, validation handling, rate limiting, error interception, and request enrichment.
models	Defines MongoDB schemas and domain entities for users, resumes, sessions, analytics, and admin activity.
routes	Maps HTTP verbs and URLs to controller handlers and applies

	feature-specific middleware stacks.
services	Contains core business rules, AI orchestration logic, document processing, and domain workflows.
repositories	Encapsulates data-access logic to keep database querying patterns consistent and testable.
validations	Holds request schemas for body, params, query validation, and business-rule assertions.
prompts	Stores structured Gemini prompts, instruction templates, and system guidance for AI output consistency.
utils	Provides helper functions for formatting, token operations, response shaping, and reusable internal tools.
jobs	Supports asynchronous or scheduled tasks such as analytics aggregation, cleanup, and report generation.
constants	Contains centralized enums, status labels, route keys, and configuration defaults.
tests	Houses unit, integration, contract, and regression tests for backend modules.
docs	Keeps internal API references, prompt notes, and engineering support documentation.

Database Design

The data model is centered on the user as the root entity. Resumes, ATS reports, job matches, interview sessions, roadmaps, coach conversations, analytics, and administrative actions all relate directly or indirectly to a single authenticated identity. MongoDB is well suited to this product because several records, especially AI-generated outputs, contain flexible nested structures that evolve over time.

Collections Overview

Collection	Purpose
users	Stores authenticated user identity, profile metadata, preferences, readiness context, and access role.
resumes	Stores resume drafts, imported resumes, parsed structures, and version metadata for each user.
atsReports	Captures ATS analysis runs, issue lists, keyword findings, and recommendation history.
jobMatches	Stores resume-versus-job-description comparison outputs and target-role fit evidence.
interviewSessions	Stores mock interview sessions, questions, answers, transcripts, and evaluation outputs.
roadmaps	Stores personalized growth plans, milestones, deadlines, and completion states.
coachConversations	Persists conversational AI coaching threads and saved insight references.
analyticsEvents	Captures product events for dashboard metrics, funnel reporting, and operational insights.
adminLogs	Maintains an auditable record of privileged admin actions and governance events.

users

Stores authenticated user identity, profile metadata, preferences, readiness context, and access role.

Field	Type	Description
_id	ObjectId	Primary key
fullName	String	User display name
email	String	Unique login identifier
passwordHash	String	bcrypt hashed password
role	String	student or admin
isVerified	Boolean	Email verification flag
targetRole	String	Desired career role
skills	Array[String]	Declared user skills
profileCompleteness	Number	Computed readiness percentage
createdAt	Date	Creation timestamp

Indexes

- unique(email)
- index(role)
- index(targetRole)

resumes

Stores resume drafts, imported resumes, parsed structures, and version metadata for each user.

Field	Type	Description
_id	ObjectId	Primary key
userId	ObjectId	Reference to users
title	String	Resume label
sourceType	String	builder or uploaded
sections	Object	Structured resume content
templateKey	String	Applied visual template
currentScore	Number	Latest composite quality score
fileUrl	String	Optional stored file link
version	Number	Draft version number
updatedAt	Date	Last modification timestamp

Indexes

- index(userId)
- compound(userId, updatedAt)

- `text(sections.summary, sections.skills)`

atsReports

Captures ATS analysis runs, issue lists, keyword findings, and recommendation history.

Field	Type	Description
<code>_id</code>	ObjectId	Primary key
<code>userId</code>	ObjectId	Reference to users
<code>resumeId</code>	ObjectId	Reference to resumes
<code>score</code>	Number	Overall ATS score
<code>keywordsMatched</code>	Array[String]	Detected matched keywords
<code>keywordsMissing</code>	Array[String]	Missing important terms
<code>issues</code>	Array[Object]	Formatting and section issues
<code>suggestions</code>	Array[String]	Improvement recommendations
<code>createdAt</code>	Date	Analysis timestamp

Indexes

- `index(userId)`
- `index(resumeId)`
- `compound(userId, createdAt)`

jobMatches

Stores resume-versus-job-description comparison outputs and target-role fit evidence.

Field	Type	Description
<code>_id</code>	ObjectId	Primary key
<code>userId</code>	ObjectId	Reference to users
<code>resumeId</code>	ObjectId	Reference to resumes
<code>jobTitle</code>	String	Analyzed target title
<code>company</code>	String	Optional company name
<code>matchScore</code>	Number	Overall alignment score
<code>matchedSkills</code>	Array[String]	Detected aligned skills
<code>missingSkills</code>	Array[String]	Uncovered gaps
<code>recommendations</code>	Array[String]	Improvement actions
<code>createdAt</code>	Date	Comparison timestamp

Indexes

- index(userId)
- text(jobTitle, company)
- compound(userId, createdAt)

InterviewSessions

Stores mock interview sessions, questions, answers, transcripts, and evaluation outputs.

Field	Type	Description
_id	ObjectId	Primary key
userId	ObjectId	Reference to users
roleTarget	String	Target interview role
difficulty	String	easy, medium, hard
questions	Array[Object]	Generated question set
answers	Array[Object]	Submitted answers or transcripts
scores	Object	Criterion-wise evaluation
feedback	Array[String]	Narrative coaching output
status	String	draft, active, completed
createdAt	Date	Session timestamp

Indexes

- index(userId)
- index(status)
- compound(userId, createdAt)

roadmaps

Stores personalized growth plans, milestones, deadlines, and completion states.

Field	Type	Description
_id	ObjectId	Primary key
userId	ObjectId	Reference to users
targetRole	String	Roadmap destination role
skillGaps	Array[String]	Detected missing capabilities
milestones	Array[Object]	Planned milestones with

		steps
timelineWeeks	Number	Planned duration
progressPercent	Number	Roadmap completion status
resources	Array[Object]	Learning resources
createdAt	Date	Creation timestamp

Indexes

- index(userId)
- index(targetRole)

coachConversations

Persists conversational AI coaching threads and saved insight references.

Field	Type	Description
_id	ObjectId	Primary key
userId	ObjectId	Reference to users
threadTitle	String	Conversation label
messages	Array[Object]	Ordered chat messages
contextRefs	Array[String]	Attached module context references
savedInsights	Array[String]	Pinned advice items
updatedAt	Date	Latest interaction timestamp

Indexes

- index(userId)
- compound(userId, updatedAt)

analyticsEvents

Captures product events for dashboard metrics, funnel reporting, and operational insights.

Field	Type	Description
_id	ObjectId	Primary key
userId	ObjectId	Optional reference to users
eventType	String	Normalized event name
module	String	Associated feature domain
metadata	Object	Flexible event payload

createdAt	Date	Event timestamp

Indexes

- compound(eventType, createdAt)
- index(module)
- ttl(optional cleanup for low-value raw events)

adminLogs

Maintains an auditable record of privileged admin actions and governance events.

Field	Type	Description
_id	ObjectId	Primary key
adminId	ObjectId	Reference to users
action	String	Action type
targetType	String	Affected entity
targetId	ObjectId	Affected record id
reason	String	Optional business reason
createdAt	Date	Audit timestamp

Indexes

- index(adminId)
- compound(action, createdAt)

Relationships

Source	Relationship	Target
users	1 to many	resumes
users	1 to many	atsReports
users	1 to many	jobMatches
users	1 to many	interviewSessions
users	1 to many	roadmaps
users	1 to many	coachConversations
users	1 to many	analyticsEvents
users(admin)	1 to many	adminLogs
resumes	1 to many	atsReports and jobMatches

ER Diagram

The following text-based ER representation communicates the logical ownership model of the platform. It emphasizes that all decision-support modules are anchored to user identity and can therefore be aggregated into longitudinal analytics and coaching context.

```
[users] 1-----* [resumes] 1-----* [atsReports]
|
|               |
|               +-----* [jobMatches]
|
+-----* [interviewSessions]
|
+-----* [roadmaps]
|
+-----* [coachConversations]
|
+-----* [analyticsEvents]
|
+-----* [adminLogs] (when role = admin)
```

Note: Document-style ER diagrams are intentionally simplified here. During implementation, these relationships should also be reflected in schema validation, ownership checks, and cascade-handling policies.

API Documentation

The REST API design follows predictable resource naming, explicit versioning, JWT-based protection, and controller-service separation. All endpoints must return structured JSON with standardized success and error envelopes so frontend state management remains deterministic and easy to test.

Authentication APIs

Method	URL	Authentication	Description
POST	/api/v1/auth/register	No	Create a new user account and persist profile defaults.
POST	/api/v1/auth/verify-email	No	Verify account ownership using OTP or token.
POST	/api/v1/auth/login	No	Authenticate user and issue access and refresh tokens.
POST	/api/v1/auth/refresh	No	Rotate access token using a valid refresh token.
POST	/api/v1/auth/logout	Yes	Invalidate active refresh token and close the session.

POST /api/v1/auth/register

Specification	Details
Method	POST
URL	/api/v1/auth/register
Authentication	No
Description	Create a new user account and persist profile defaults.
Request Body	fullName, email, password, targetRole
Validation	Valid email, strong password, unique email
Success Response	201 Created with user profile and verification status
Common Errors	400 validation error, 409 email exists

Example	<code>{"fullName":"Aarav Shah","email":"aarav@example.com","password":"StrongPass@123","targetRole":"Frontend Developer"}</code>

POST /api/v1/auth/verify-email

Specification	Details
Method	POST
URL	/api/v1/auth/verify-email
Authentication	No
Description	Verify account ownership using OTP or token.
Request Body	email, token
Validation	Token required and not expired
Success Response	200 Verified account response
Common Errors	400 invalid token, 410 expired token
Example	<code>{"email":"aarav@example.com","token":"493281"}</code>

POST /api/v1/auth/login

Specification	Details
Method	POST
URL	/api/v1/auth/login
Authentication	No
Description	Authenticate user and issue access and refresh tokens.
Request Body	email, password
Validation	Existing user, verified account, password match
Success Response	200 Token pair and profile summary
Common Errors	401 invalid credentials, 403 unverified account
Example	<code>{"email":"aarav@example.com","password":"StrongPass@123"}</code>

POST /api/v1/auth/refresh

Specification	Details
Method	POST
URL	/api/v1/auth/refresh
Authentication	No

Description	Rotate access token using a valid refresh token.
Request Body	refreshToken
Validation	Signed token, not revoked, not expired
Success Response	200 New access token
Common Errors	401 invalid refresh token
Example	{"refreshToken":"<refresh-token>"}

POST /api/v1/auth/logout

Specification	Details
Method	POST
URL	/api/v1/auth/logout
Authentication	Yes
Description	Invalidate active refresh token and close the session.
Request Body	refreshToken
Validation	Valid session token
Success Response	200 Logout confirmation
Common Errors	401 unauthorized
Example	{"refreshToken":"<refresh-token>"}

Profile APIs

Method	URL	Authentication	Description
GET	/api/v1/users/me	Yes	Fetch the authenticated user's profile and readiness summary.
PUT	/api/v1/users/me	Yes	Update personal, academic, and preference information.
PUT	/api/v1/users/password	Yes	Rotate account password securely.

GET /api/v1/users/me

Specification	Details
Method	GET
URL	/api/v1/users/me

Authentication	Yes
Description	Fetch the authenticated user's profile and readiness summary.
Request Body	None
Validation	Valid JWT
Success Response	200 User profile payload
Common Errors	401 unauthorized
Example	{ }

PUT /api/v1/users/me

Specification	Details
Method	PUT
URL	/api/v1/users/me
Authentication	Yes
Description	Update personal, academic, and preference information.
Request Body	profile fields
Validation	Field types, lengths, enum validation
Success Response	200 Updated profile
Common Errors	400 validation error
Example	{"fullName":"Aarav Shah","targetRole":"Software Engineer","skills":["React","Node.js"]}

PUT /api/v1/users/password

Specification	Details
Method	PUT
URL	/api/v1/users/password
Authentication	Yes
Description	Rotate account password securely.
Request Body	currentPassword, newPassword
Validation	Password policy and current password match
Success Response	200 Password updated
Common Errors	400 weak password, 401 mismatch
Example	{"currentPassword":"OldPass@123","newPassword":"NewPass@123"}

Resume APIs

Method	URL	Authentication	Description
GET	/api/v1/resumes	Yes	List all resumes

			owned by the current user.
POST	/api/v1/resumes	Yes	Create a new resume draft using form content or AI seed input.
POST	/api/v1/resumes/parse	Yes	Upload and parse an existing resume into editable structure.
PUT	/api/v1/resumes/:id	Yes	Update an existing resume draft or parsed document.
POST	/api/v1/resumes/:id/ai-write	Yes	Generate AI suggestions for summary, bullets, or skill phrasing.

GET /api/v1/resumes

Specification	Details
Method	GET
URL	/api/v1/resumes
Authentication	Yes
Description	List all resumes owned by the current user.
Request Body	None
Validation	Valid JWT
Success Response	200 Array of resumes
Common Errors	401 unauthorized
Example	{ }

POST /api/v1/resumes

Specification	Details
Method	POST
URL	/api/v1/resumes
Authentication	Yes
Description	Create a new resume draft using form content or AI seed input.
Request Body	title, sections, templateKey
Validation	Required title, normalized sections
Success Response	201 Resume created

Common Errors	400 invalid body
Example	<code>{"title":"SDE Resume","templateKey":"modern-blue","sections":{"summary":"..."}}</code>

POST /api/v1/resumes/parse

Specification	Details
Method	POST
URL	/api/v1/resumes/parse
Authentication	Yes
Description	Upload and parse an existing resume into editable structure.
Request Body	multipart file
Validation	PDF or DOCX file, size limits
Success Response	200 Parsed sections and confidence details
Common Errors	415 unsupported format, 422 parse failure
Example	<code>{"file":"resume.pdf"}</code>

PUT /api/v1/resumes/:id

Specification	Details
Method	PUT
URL	/api/v1/resumes/:id
Authentication	Yes
Description	Update an existing resume draft or parsed document.
Request Body	sections, templateKey
Validation	Ownership, valid section schema
Success Response	200 Resume updated
Common Errors	404 not found
Example	<code>{"sections":{"experience":[{"company":"ABC","impact":"..."}]}}</code>

POST /api/v1/resumes/:id/ai-write

Specification	Details
Method	POST
URL	/api/v1/resumes/:id/ai-write
Authentication	Yes
Description	Generate AI suggestions for summary, bullets, or skill phrasing.

Request Body	promptContext, targetRole
Validation	Ownership and supported section types
Success Response	200 Suggested content
Common Errors	429 AI rate limit, 502 upstream failure
Example	<pre>{"promptContext":"Improve summary for backend role","targetRole":"Backend Developer"}</pre>

ATS and Match APIs

Method	URL	Authentication	Description
POST	/api/v1/ats/analyze	Yes	Analyze ATS compatibility for a selected resume.
POST	/api/v1/job-match	Yes	Compare a resume against a job description.

POST /api/v1/ats/analyze

Specification	Details
Method	POST
URL	/api/v1/ats/analyze
Authentication	Yes
Description	Analyze ATS compatibility for a selected resume.
Request Body	resumeld, targetRole, jobDescription(optional)
Validation	Resume ownership, valid identifier
Success Response	200 ATS report with score and issues
Common Errors	404 resume not found, 422 invalid content
Example	<pre>{"resumeld":"<resumeld>","targetRole":"Data Analyst"}</pre>

POST /api/v1/job-match

Specification	Details
Method	POST
URL	/api/v1/job-match
Authentication	Yes
Description	Compare a resume against a job

	description.
Request Body	resumeld, jobDescription, jobTitle, company(optional)
Validation	Resume ownership, non-empty JD
Success Response	200 Match report with gaps and recommendations
Common Errors	400 empty JD, 404 resume missing
Example	{ "resumeld": "<resumeld>", "jobTitle": "Frontend Developer", "jobDescription": "We need React, TypeScript, REST..." }

Interview APIs

Method	URL	Authentication	Description
POST	/api/v1/interviews	Yes	Create a new mock interview session.
PUT	/api/v1/interviews/:id/answer	Yes	Submit an answer for a specific interview question.
POST	/api/v1/interviews/:id/evaluate	Yes	Evaluate completed answers and generate final report.
GET	/api/v1/interviews/:id/report	Yes	Fetch the detailed interview report for review UI.

POST /api/v1/interviews

Specification	Details
Method	POST
URL	/api/v1/interviews
Authentication	Yes
Description	Create a new mock interview session.
Request Body	roleTarget, difficulty, interviewType
Validation	Supported enums and valid JWT
Success Response	201 Interview session with generated questions
Common Errors	400 invalid setup
Example	{ "roleTarget": "Software Engineer", "difficulty": "medium", "inter

	viewType:"technical"}
--	-----------------------

PUT /api/v1/interviews/:id/answer

Specification	Details
Method	PUT
URL	/api/v1/interviews/:id/answer
Authentication	Yes
Description	Submit an answer for a specific interview question.
Request Body	questionId, answerText
Validation	Ownership and active session
Success Response	200 Saved answer
Common Errors	404 session missing, 409 session completed
Example	{"questionId":"q1","answerText":"My approach would be..."}

POST /api/v1/interviews/:id/evaluate

Specification	Details
Method	POST
URL	/api/v1/interviews/:id/evaluate
Authentication	Yes
Description	Evaluate completed answers and generate final report.
Request Body	None or evaluation options
Validation	Completed question set
Success Response	200 Scorecard and improvement report
Common Errors	422 insufficient answers, 502 AI error
Example	{}

GET /api/v1/interviews/:id/report

Specification	Details
Method	GET
URL	/api/v1/interviews/:id/report
Authentication	Yes
Description	Fetch the detailed interview report for review UI.
Request Body	None
Validation	Ownership validation
Success Response	200 Full report payload

Common Errors	404 report missing
Example	{}

Roadmap and Coach APIs

Method	URL	Authentication	Description
POST	/api/v1/roadmaps	Yes	Generate a personalized roadmap from profile and performance context.
PUT	/api/v1/roadmaps/:id/progress	Yes	Update milestone completion status.
POST	/api/v1/coach/chat	Yes	Generate a contextual coaching response.

POST /api/v1/roadmaps

Specification	Details
Method	POST
URL	/api/v1/roadmaps
Authentication	Yes
Description	Generate a personalized roadmap from profile and performance context.
Request Body	targetRole, timeAvailability(optional)
Validation	Profile completeness threshold
Success Response	201 Roadmap document
Common Errors	400 insufficient profile, 502 AI failure
Example	{ "targetRole": "Full Stack Developer", "timeAvailability": "10 hours/week" }

PUT /api/v1/roadmaps/:id/progress

Specification	Details
Method	PUT
URL	/api/v1/roadmaps/:id/progress
Authentication	Yes
Description	Update milestone completion status.
Request Body	milestoneId, status

Validation	Ownership and valid status
Success Response	200 Updated roadmap progress
Common Errors	404 roadmap not found
Example	<code>{"milestoneId":"m2","status":"completed"}</code>

POST /api/v1/coach/chat

Specification	Details
Method	POST
URL	/api/v1/coach/chat
Authentication	Yes
Description	Generate a contextual coaching response.
Request Body	message, contextRefs(optional)
Validation	Message length, JWT, supported context keys
Success Response	200 Assistant reply and saved thread info
Common Errors	429 rate limit, 502 AI error
Example	<code>{"message":"How do I improve for backend interviews?","contextRefs":["latestInterview","currentRoadmap"]}</code>

Analytics and Admin APIs

Method	URL	Authentication	Description
GET	/api/v1/analytics/dashboard	Yes	Return dashboard metrics for the authenticated user.
GET	/api/v1/admin/users	Admin	List users with filters, pagination, and role data.
PUT	/api/v1/admin/users/:id/role	Admin	Promote or demote user access roles.
GET	/api/v1/admin/metrics	Admin	Fetch platform-level operational and usage analytics.

GET /api/v1/analytics/dashboard

Specification	Details
Method	GET
URL	/api/v1/analytics/dashboard
Authentication	Yes
Description	Return dashboard metrics for the authenticated user.
Request Body	None
Validation	Valid JWT
Success Response	200 Cards, charts, and recent activity
Common Errors	401 unauthorized
Example	{ }

GET /api/v1/admin/users

Specification	Details
Method	GET
URL	/api/v1/admin/users
Authentication	Admin
Description	List users with filters, pagination, and role data.
Request Body	query params
Validation	Admin role required
Success Response	200 Paginated users
Common Errors	403 forbidden
Example	{ }

PUT /api/v1/admin/users/:id/role

Specification	Details
Method	PUT
URL	/api/v1/admin/users/:id/role
Authentication	Admin
Description	Promote or demote user access roles.
Request Body	role
Validation	Allowed roles only
Success Response	200 Updated user role
Common Errors	400 invalid role, 403 forbidden
Example	{ "role": "admin" }

GET /api/v1/admin/metrics

Specification	Details
---------------	---------

Method	GET
URL	/api/v1/admin/metrics
Authentication	Admin
Description	Fetch platform-level operational and usage analytics.
Request Body	None
Validation	Admin role required
Success Response	200 Aggregated metrics
Common Errors	403 forbidden
Example	{ }

Tip: Production APIs should use request IDs, structured logging, and consistent response envelopes so errors can be traced across frontend, backend, and AI upstream calls.

Authentication Flow

This workflow documents the operating sequence, state transitions, and decision points for the authentication flow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

19. User opens signup or login route from the public landing page.
20. Frontend validates email, password, and profile inputs before network submission.
21. Backend validates payload, checks account status, hashes password on registration, and issues tokens on successful login.
22. Refresh token is stored securely according to deployment strategy, while access token is attached to protected API calls.
23. Protected route middleware and React Query hydration load dashboard context after authentication succeeds.

Workflow Diagram

```
[User] -> [Signup/Login Form] -> [Express Auth Controller] -> [Validation +  
bcrypt + JWT]  
                                     -> [MongoDB users]  
                                     -> [Access Token + Refresh Token]  
                                     -> [Protected Dashboard Session]
```

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

Resume Builder Workflow

This workflow documents the operating sequence, state transitions, and decision points for the resume builder workflow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

24. User creates a new draft or selects a starter template from the resume hub.
25. Section-based forms collect profile, education, experience, projects, skills, and certification data.
26. AI assistance can generate summaries, bullet points, and achievement wording using role-aware prompts.
27. The live preview reflects updates immediately and the user saves versioned drafts to MongoDB.
28. The saved resume becomes reusable input for ATS, job-match, and interview preparation modules.

Workflow Diagram

[Resume Form] -> [Structured Sections] -> [Gemini Suggestions] -> [Preview Engine]
-> [Save Draft / Version] -> [Resume Repository]

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

Resume Parser Workflow

This workflow documents the operating sequence, state transitions, and decision points for the resume parser workflow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

29. User uploads a PDF or DOCX resume through the parser intake interface.
30. Backend validates file type and size, then extracts readable text and basic document structure.
31. Parsing service maps extracted text into resume entities such as summary, experience, education, projects, and skills.
32. Confidence rules identify uncertain fields so the frontend can request user confirmation before final save.
33. The reconstructed data is saved as an editable resume draft for downstream analysis and editing.

Workflow Diagram

[Upload Resume] -> [File Validation] -> [Text Extraction] -> [Section Mapping]
-> [Confidence Review] ->
[Editable Draft Saved]

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

ATS Workflow

This workflow documents the operating sequence, state transitions, and decision points for the ats workflow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

34. User selects a resume and optionally supplies a target role or job description context.
35. The ATS service evaluates keyword presence, formatting quality, section completeness, and structural consistency.
36. Gemini-enhanced reasoning converts low-level findings into prioritized recommendations understandable by non-expert users.
37. The report is persisted for trend analysis and surfaced in the ATS view as cards, tables, and score visuals.
38. Users can push recommended edits back into the resume editing workflow for remediation.

Workflow Diagram

```
[Resume Selection] -> [ATS Analysis Engine] -> [Score + Issues +  
Recommendations]  
                                     -> [Persist Report] -> [UI  
Diagnostics + Improve Actions]
```

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

Resume vs JD Workflow

This workflow documents the operating sequence, state transitions, and decision points for the resume vs jd workflow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

39. User chooses a stored resume and submits a target job description.
40. Matching service extracts requirements, role terminology, skill expectations, and implied experience signals.
41. The resume content is compared against these signals to calculate a match score and identify aligned versus missing evidence.
42. Recommendations are prioritized so the user knows which edits produce the greatest increase in job relevance.
43. The saved result supports application strategy, targeted editing, and historical comparison across multiple roles.

Workflow Diagram

```
[Resume + JD] -> [Requirement Extraction] -> [Semantic + Keyword Match]
                                     -> [Gap Analysis] -> [Prioritized
Improvement Plan]
```

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

Interview Workflow

This workflow documents the operating sequence, state transitions, and decision points for the interview workflow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

44. User configures interview type, target role, and difficulty level.
45. Gemini generates contextual questions based on role expectations and optionally the user's resume context.
46. The user submits typed or transcribed answers while session state tracks progress and timing.
47. Evaluation service scores answers for relevance, structure, confidence, and technical depth.
48. A detailed report is created and can influence roadmap generation and coaching conversations.

Workflow Diagram

```
[Interview Setup] -> [Question Generation] -> [Answer Capture] ->
[Evaluation Engine]                                     -> [Detailed Report] -> [Roadmap
/ Coach Context]
```

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

Career Roadmap Workflow

This workflow documents the operating sequence, state transitions, and decision points for the career roadmap workflow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

- 49. Roadmap generation starts from profile data, target role, ATS insights, and interview performance patterns.
- 50. The system identifies high-priority gaps and organizes them into milestones that balance urgency and feasibility.
- 51. Each milestone is translated into action items, suggested resources, and success criteria.
- 52. Users track completion over time, creating a continuously updated progress signal for the dashboard.
- 53. The roadmap may be regenerated when new interviews, target roles, or score improvements change the user's direction.

Workflow Diagram

[Profile + Scores + Goals] -> [Gap Prioritization] -> [Milestone Generation]
-> [Resource Plan] -> [Track Progress]

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

Career Coach Workflow

This workflow documents the operating sequence, state transitions, and decision points for the career coach workflow. The goal is to ensure that the implementation remains predictable, observable, and easy to maintain as the platform grows.

54. User opens the coaching interface and submits a context-aware question or planning request.
55. Frontend optionally sends linked context such as latest ATS report, interview performance, or roadmap status.
56. Coaching service assembles prompt context, applies safety and scope rules, and requests a Gemini response.
57. The response is displayed in conversational format and important insights can be pinned for later reuse.
58. Conversation history is stored so future coaching remains personalized and longitudinal rather than isolated.

Workflow Diagram

```
[User Prompt] + [Context References] -> [Prompt Assembly] -> [Gemini Coach Response]
                                                    -> [Save Thread + Insights]
```

Note: Each workflow should emit analytics events at major milestones so product teams can measure abandonment, completion, and performance trends.

Dashboard Analytics

Dashboard analytics should transform raw activity into clear decision-support signals. Rather than showing metrics for their own sake, each widget must answer an action-oriented question such as whether the resume is improving, whether the user is becoming more interview-ready, or whether the roadmap is progressing on schedule.

Metric	Meaning
Resume Quality Score	Composite score generated from resume structure, content strength, and optimization level.
ATS Readiness	Most recent ATS score with delta versus previous analysis.
Job Match Average	Average alignment across saved target roles and job descriptions.
Interview Performance	Average score from completed mock interviews with trend direction.
Roadmap Progress	Milestone completion percentage and overdue-action count.
Engagement Frequency	Number of meaningful weekly actions such as edits, analyses, or sessions.

Example Analytics Composition

Resume Score Trend

ATS Readiness Trend

Interview Improvement

Roadmap Completion

:

:

:

:



Tip: Analytics cards should always pair a metric with contextual interpretation, such as trend direction, comparison window, or recommended next action.

Admin Panel

The admin panel is the operational control center of the platform. It must provide visibility into user adoption, system health, content safety, AI usage patterns, and privileged interventions, while simultaneously enforcing strict authorization and auditability.

Admin Capability	Professional Expectation
User Management	Search, filter, inspect, and update user roles or statuses with audit logging.
Usage Analytics	Review active users, funnel conversion, feature adoption, and AI request volume.
Support and Moderation	Investigate anomalous user behavior, content complaints, or support escalations.
Platform Health	Observe failure trends, slow endpoints, and AI provider instability.
Governance	Maintain traceability for role changes, suspensions, exports, and administrative decisions.

Security

Security is a first-class architectural requirement because the platform handles user identity, resumes, interview transcripts, and AI-generated guidance. The security design must protect confidentiality, preserve data integrity, and ensure that privileged operations remain verifiable and restricted.

Control	Layer	Implementation Rationale
JWT Access Token	API Security	Short-lived bearer token protects API routes and limits long-lived credential exposure.
Refresh Token	Session Continuity	Stored and rotated securely to renew access without frequent re-login.
Helmet	HTTP Hardening	Sets protective headers for clickjacking, MIME sniffing, and transport-related defenses.
Rate Limiting	Abuse Prevention	Protects login, AI, and public endpoints from brute force and cost-amplification misuse.
Validation	Input Safety	Schema-level validation blocks malformed requests and enforces business rules early.
Environment Variables	Secret Management	Keeps API keys, JWT secrets, and database credentials outside source code.
CORS	Origin Control	Restricts browser access to approved frontend origins and environments.
XSS Protection	Frontend and API	Output encoding, sanitized rendering, and safe component practices reduce script injection risk.
NoSQL Injection Defense	Database Safety	Query sanitization and

		strict schema handling reduce malicious operator injection.
Role-Based Access	Authorization	Admin-only routes and privileged actions are checked server-side for every request.

Warning: JWT secrets, refresh token secrets, and AI credentials must be rotated through environment management and must never be hard-coded into frontend bundles or source repositories.

Performance Optimization

Performance optimization should be approached holistically across bundle delivery, request orchestration, database access, AI response management, and perceived user experience. In an AI-enabled product, optimization also includes minimizing unnecessary prompt invocations and communicating clearly when long-running tasks are in progress.

Technique	Layer	Implementation Outcome
Lazy Loading	Frontend	Load heavy routes such as interview reports and admin views only when needed.
Pagination	Frontend and API	Reduce data transfer and render cost for resumes, interviews, and admin tables.
Skeleton Loading	Frontend UX	Improve perceived performance while preserving layout stability during network calls.
Image Compression	Media	Optimize uploaded profile and branding assets before delivery.
PDF Compression	Documents	Reduce resume export payload sizes for faster storage and download.
Caching	Client and Server	Use React Query cache and selective backend caching for repeat-intensive reads.
Code Splitting	Frontend Bundle	Reduce initial bundle size and improve first-load performance.
Debounce	Forms and Search	Prevent redundant API calls while users type into search and prompt fields.
Virtualization	Large Lists	Render only visible rows in admin and analytics-heavy grids.
Optimized Queries	Backend	Project only required

		fields and avoid expensive uncontrolled aggregations.
Indexes	Database	Accelerate frequent lookups for user, resume, report, and analytics paths.

Error Handling

A production-grade system must fail gracefully. Errors should be captured early, translated consistently, logged with context, and surfaced to the user in a way that encourages recovery rather than confusion. The goal is not merely to catch exceptions, but to preserve trust during failure scenarios.

Capability	Layer	Professional Behavior
Global Error Handler	Backend	Central middleware normalizes operational and unexpected failures into consistent JSON responses.
React Error Boundary	Frontend	Prevents full-app crashes by isolating rendering failures and showing recovery UI.
404 Handling	Routing	Unknown pages and missing resources return dedicated not-found states.
500 Handling	Runtime	Server-side exceptions are logged, masked, and returned with safe messaging.
Retry Button	UX Recovery	Critical failed views expose explicit retry actions for user-controlled recovery.
Toast Messages	Feedback	Short-lived notifications communicate success, warning, and failure events clearly.
Logging	Observability	Structured logs preserve stack traces, request context, and AI failure diagnostics.
Validation Errors	Input Feedback	Field-level and summary messages help users correct invalid data quickly.

Deployment Guide

Deployment is structured around a decoupled frontend-backend-cloud model. The React client is deployed to Vercel for fast global delivery, the Express API is deployed to Render for managed backend hosting, and MongoDB Atlas and ImageKit provide managed data and asset services. This arrangement is suitable for startup-stage SaaS operations while still supporting later scale transitions.

Deployment Sequence

59. Provision MongoDB Atlas cluster, database user, IP access policy, and connection string.
60. Create ImageKit credentials and configure upload, transformation, and delivery endpoints.
61. Deploy backend to Render with environment variables, health checks, build command, and secure origin configuration.
62. Deploy frontend to Vercel with production API base URL and environment-aligned feature flags.
63. Configure DNS, HTTPS, monitoring, and post-deployment smoke tests before public release.

Environment Variables

Variable	Layer	Purpose
CLIENT_API_BASE_URL	Frontend	Base URL of backend REST service
JWT_SECRET	Backend	Secret for signing access tokens
JWT_REFRESH_SECRET	Backend	Secret for signing refresh tokens
MONGODB_URI	Backend	MongoDB Atlas connection string
GEMINI_API_KEY	Backend	Google Gemini API credential
IMAGEKIT_PUBLIC_KEY	Backend	ImageKit public key
IMAGEKIT_PRIVATE_KEY	Backend	ImageKit private key
IMAGEKIT_URL_ENDPOINT	Backend	ImageKit delivery endpoint
CORS_ORIGIN	Backend	Allowed frontend origins
NODE_ENV	Backend	Runtime environment

		flag
--	--	------

Production Checklist

- Verify JWT and refresh token secret rotation policy.
- Confirm CORS origin restrictions for all environments.
- Enable structured logging and alerting for critical failures.
- Run database backup and restore validation.
- Check AI usage quotas, rate limits, and fallback messaging.
- Validate file upload restrictions and malware-safe handling assumptions.
- Confirm all critical routes have loading, error, and empty states.

Testing Strategy

A robust testing strategy is required because AI-assisted systems can fail in ways that are both deterministic and probabilistic. The platform therefore needs layered testing that validates UI behavior, API correctness, authorization rules, database queries, prompt orchestration, and resilience under partial failure conditions.

Testing Type	Primary Focus	Professional Objective
Unit Testing	Component and service logic	Validate helper functions, reducers, validators, and domain services in isolation.
Integration Testing	API + database + middleware	Verify end-to-end behavior of routes, persistence, and authorization rules.
Frontend Interaction Testing	Pages and critical flows	Test forms, route guards, loading states, and optimistic updates.
End-to-End Testing	User journey	Automate signup, resume creation, ATS run, interview, roadmap, and logout flows.
Security Testing	Auth and input vectors	Check brute-force resistance, token validation, permission leaks, and injection safety.
Performance Testing	API and query load	Measure latency under realistic concurrency and AI request volume assumptions.
Regression Testing	Release safety	Protect existing functionality whenever AI prompts, schemas, or UI flows change.
User Acceptance Testing	Product fit	Validate that user-facing workflows are understandable, complete, and outcome-oriented.

Note: AI-facing tests should include response schema validation, timeout

handling, and prompt regression checks to preserve quality when prompts evolve.

Future Scope

The platform has a strong foundation for expansion into institutional, mobile, and advanced simulation use cases. Because the architecture is modular and the core data model is already aligned with progression tracking, future features can build on existing identity, resume, analytics, and coaching assets rather than starting from scratch.

- Native mobile applications for Android and iOS with push-based reminder workflows.
- Institution dashboards for placement cells to monitor cohorts and intervention effectiveness.
- Company-specific interview packs and recruiter-targeted resume calibration models.
- Voice-first interview simulation with prosody and filler-word analysis.
- Multilingual guidance and regionalized career content.
- Calendar, LMS, and job-board integrations for broader workflow automation.

Advantages

AI Career Accelerator offers strategic and operational advantages over fragmented preparation tools by consolidating workflows, preserving context, and converting user activity into personalized guidance. These advantages increase both user value and platform defensibility.

- Unifies the placement-preparation lifecycle into one product experience.
- Improves personalization by reusing data across modules rather than treating tasks independently.
- Provides measurable analytics that help users understand progress over time.
- Supports scalable SaaS deployment with secure APIs, modular architecture, and admin control.
- Transforms AI from novelty into operational assistance tied to concrete user actions.

Limitations

Like any AI-enabled SaaS system, the platform has current-release constraints that should be acknowledged transparently. Recognizing these limitations improves planning discipline and creates a realistic roadmap for future enhancement.

- AI output quality still depends on prompt design and input completeness.
- Advanced parsing accuracy may vary for heavily stylized resume formats.
- Real-time speech evaluation is outside the current documented release scope.
- Institution-specific custom workflows require additional configuration and governance controls.
- External AI and cloud providers introduce cost and dependency considerations.

Team Members

The project responsibilities are distributed to align domain ownership with the technical strengths required for a balanced SaaS implementation. This division promotes accountability while still enabling collaborative delivery across shared interfaces and integration points.

Name	Enrollment	Role
Arman Mahetar	20230905050004	Backend Development, API Development, Database Integration
Hardik Makwana	20230905050009	Backend Development, Testing, Quality Assurance
Krunal Rathod	20230905050032	Frontend Development, AI Integration, Deployment

Conclusion

AI Career Accelerator represents a production-oriented approach to career development software. By combining resume creation, ATS analysis, job matching, interview simulation, roadmap planning, coaching, analytics, and administration into a single coherent platform, it addresses a real market problem with both product depth and engineering discipline.

The architecture, workflows, UI system, database model, API design, deployment strategy, and security controls documented in this report demonstrate that the platform is not merely a conceptual prototype. It is structured as a scalable SaaS solution capable of iterative enhancement, measurable user impact, and maintainable long-term delivery.

Final Note: This document is intentionally editable and structured with Word heading styles, tables, callouts, and automatic TOC support so it can be extended during future implementation phases.